Personal Project Experience and Interests

Zhiwei(Zavier)Zhao

Computer Science and Engineering, UCSD, La Jolla, California, 92037

zhz214@ucsd.edu

Welcome to visit my personal website:

furkathertaha.github.io

and GitHub homepage:

github.com/Furkathertaha

1. Stylized 3D reconstruction

This is a unified framework for artistically stylizing 3D Gaussian Splatting (3DGS) reconstructions using single-step diffusion models. Standard 3DGS excels at photorealistic novel-view synthesis but is locked to the appearance of its training images. This project breaks that constraint by combining Delta Distillation Sampling (DDS), which steers the 3DGS optimization toward a target artistic style via diffusion-model gradients, with Difix, a pixel-level corrector that resolves the blurriness, over-saturation, and geometric floaters that score-distillation methods typically introduce. ControlNet preserves structural detail, an IP-Adapter supplies style guidance from a single reference image, and Multi-View Frequency Consistency (MVFC) enforces stable appearance across viewpoints. The full pipeline runs on a single consumer graphics card with custom CPU-offloading and optimized CUDA rasterization kernels, producing high-fidelity stylized 3D scenes, from impressionist paintings to anime aesthetics, without sacrificing much the geometric quality of the original reconstruction.

Camera Images

arbitrary style image

reconstructed 3D scene

Demo captured on continuous camera trajectory:
https://drive.google.com/drive/folders/19w2d6qgJJ749GOTv9yhF8juy1bFi3HNr?usp=drive_link
Tech Report:
https://github.com/Furkath/Stylized-3DGS-reconsrutcion-via-DDS/blob/main/style3Drecon.pdf

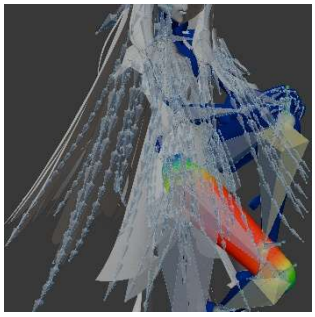
2. Fused PBR/NPR style into Monte Carlo Path Tracer

This project extends the Lajolla physically-based path tracer with a suite of non-photorealistic rendering (NPR) techniques and a character rendering pipeline, enabling stylized and physically grounded imagery to coexist in a single unified renderer.

The core contribution is integrating cel-shading into a Monte Carlo path tracer, a combination rarely attempted because NPR is traditionally a rasterization technique. Here implemented Disney Principled BSDF and extended with a custom NPR BSDF that supports toon and sphere matcap lookups, quantized cosine shading (hard-edge cel shading with configurable color bands), Fresnel-hacked specular highlights, and rim lighting. Matcap UVs are derived by projecting the shading normal into view space, allowing view-dependent stylization that still benefits from ray-traced soft shadows, indirect illumination, and global illumination, which are unavailable in a traditional rasterization NPR pipeline. The NPR and PBR material systems are unified under a single XML scene description, so any object in a scene can be assigned to either material type without pipeline changes.

Beyond shading, the project includes a character rendering pipeline covering skeletal rigging and skinning. Mesh geometry is posed from rest to animation frame via Linear Blend Skinning before being fed to the path tracer, beyond LBS future features shall support Dual Quaternion, Bulge-Free DQS, and Disney's Optimized Center of Rotation with detailed mathematical applicable to implement.
Additional renderer-level work includes volume path tracing support, a soft shadow terminator fix for low-poly models, numerical stability hardening for Windows Release builds (zero-initialization, null-pointer, and division-by-zero guards), and new light types (ambient, emission shaders).

The entire pipeline including scene authoring, character rigging, and stylized rendering runs within the extended Lajolla renderer, built with C++, CMake, and Embree for ray casting.

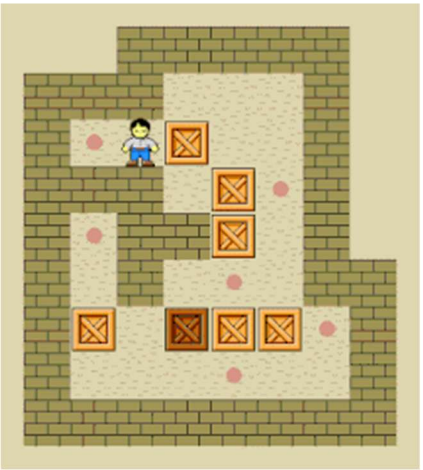
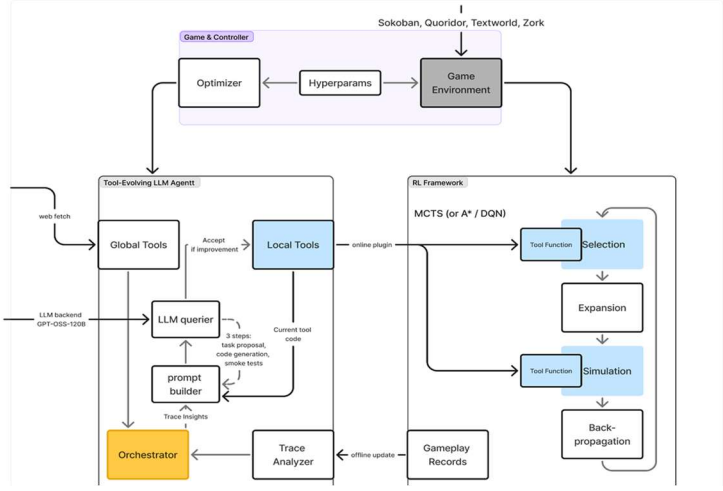


Rigging and Skinning NPR in RT Fused PBR/NPR Rendering

Report and Code Repository: https://github.com/Furkath/Lajolla_mod

3. Self-Evolving Game Agent with Tool-creation

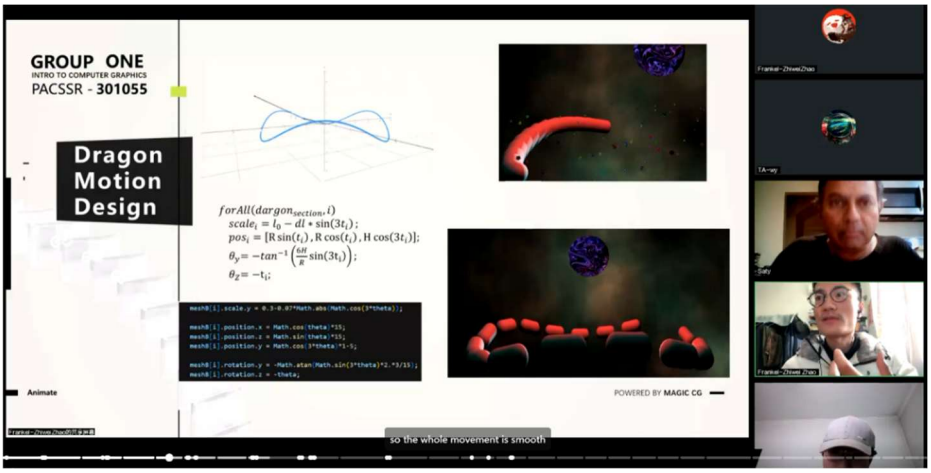
Built a self-evolving agent architecture that decouples slow LLM reasoning from fast game execution. Instead of costly per-step LLM calls during gameplay, the system uses an LLM purely offline to analyze failure traces, diagnose root causes, and generate improved Python heuristics that plug directly into MCTS/A*. The planning core runs at native speed with zero LLM overhead, while an offline evolution loop continuously refines search heuristics, accepting new code only after smoke tests and strict performance validation. The architecture is fully game-agnostic, requiring only a config swap to adapt to new domains. Evaluated on Sokoban, TextWorld, and Zork, the system achieved up to 74.4% reduction in search nodes, 100% zero-shot accuracy on unseen TextWorld maps (where DQN collapses beyond 11 rooms), and ~20% score improvement on Zork, all producing transparent, auditable code rather than opaque neural weights.

Repositroy: https://github.com/zhuyuezx/CSE291A_Project

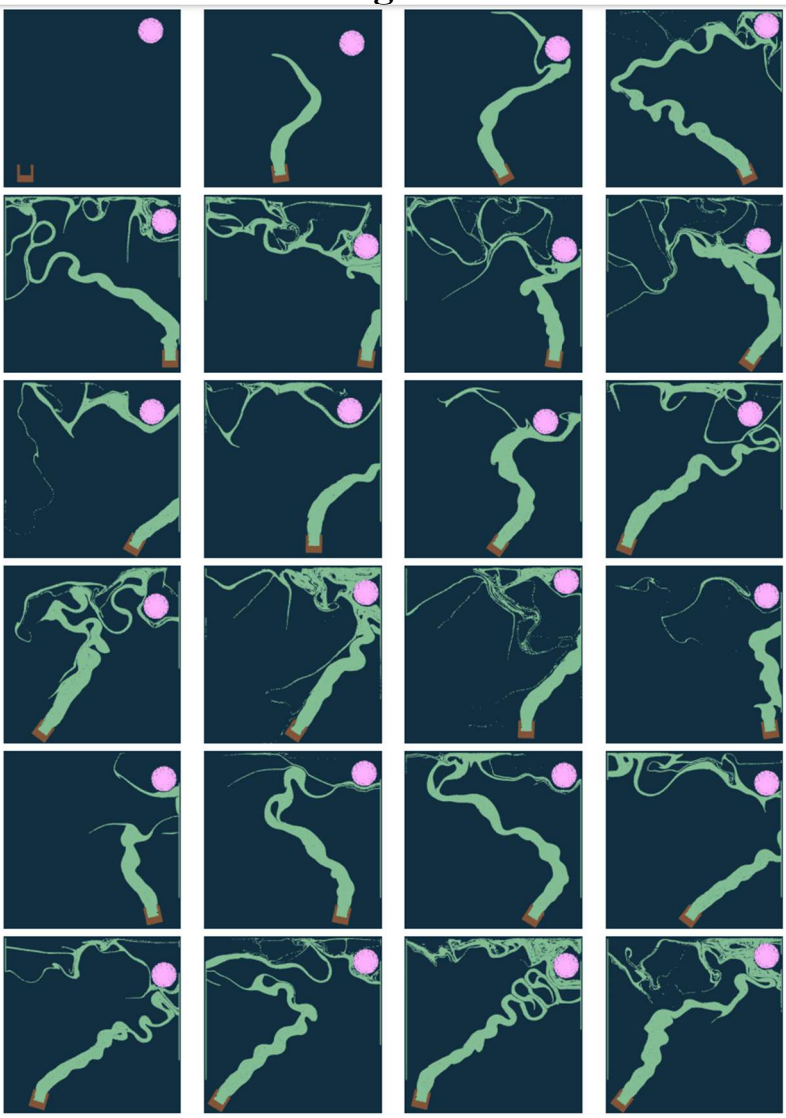
4. Web3D Online Practice

This project was remotely guided by Professor Saty Raghavachary from the Computer Science Department at the University of Southern California. It covered the history of the animation industry's development, the industrial applications of various graphics technologies, and the current development of the integration of the 3D industry with AI. The project mimicked the complete production process of Pixar's Animation Feature, and a team was formed to complete the construction of a 3D webpage. I led an international group (including students from UC Irvine and the University of Glasgow), coordinated the output of STL (no UV) models, GLSL-Shader production, and the completion of three.JS animations (model affine transformation, Vertex deformation) and the webpage.



Details of the Animation webpage: https://furkathertaha.github.io/assets/threeJS/sf/long1.html
3D webpage source code: https://github.com/Furkath/threeJS-GLSL_STL-3D-webpage-design

5. Deep Reinforcement Learning for Interactive Animation



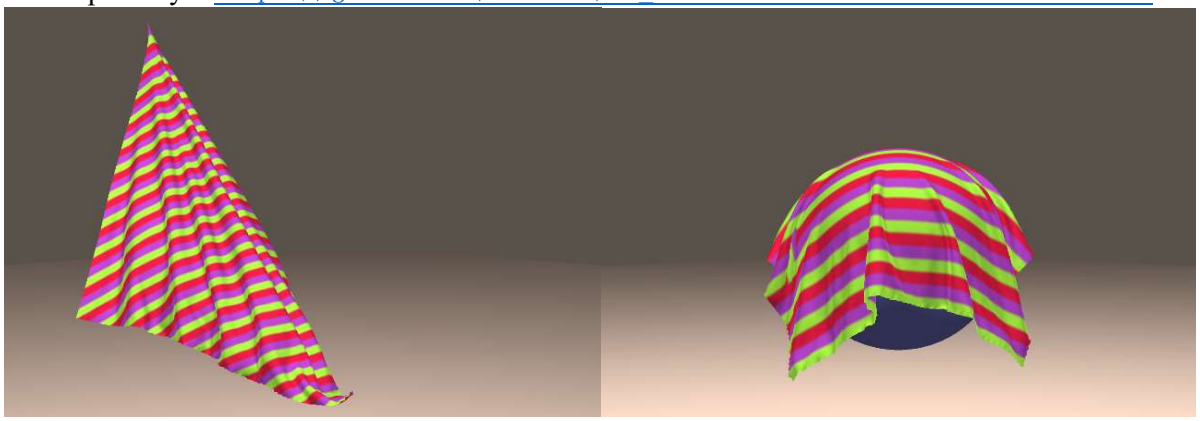
According to the theme of the Machine Learning Reproduce Challenge (MLRC) 2023, we reproduced, expanded, and optimized the deep learning application in graphics in recent years' papers published in ACM TOG. As the group leader, together with my classmates, I chose the work of controlling the position and posture of a small ball using deep reinforcement learning (Soft Actor-Critic Algorithm) in fluid-rigid body simulation as the topic and plan to submit it to the MLRC2024 conference. Currently, the optimization of 2D animation has been completed: Pytorch is used to build a small-scale AlexNet as the AutoEncoder of fluid spatial information, and the definition of Reward is improved to prevent convergence difficulties caused by over-sparsity. Taichi was fully utilized to write the interactive animation of MLS-MPM fluid and rigid body, optimize the physical process of their collision, and utilization of GPU parallel particle memory. The latent space information after simulation and structural optimization can effectively improve the training speed of the model.

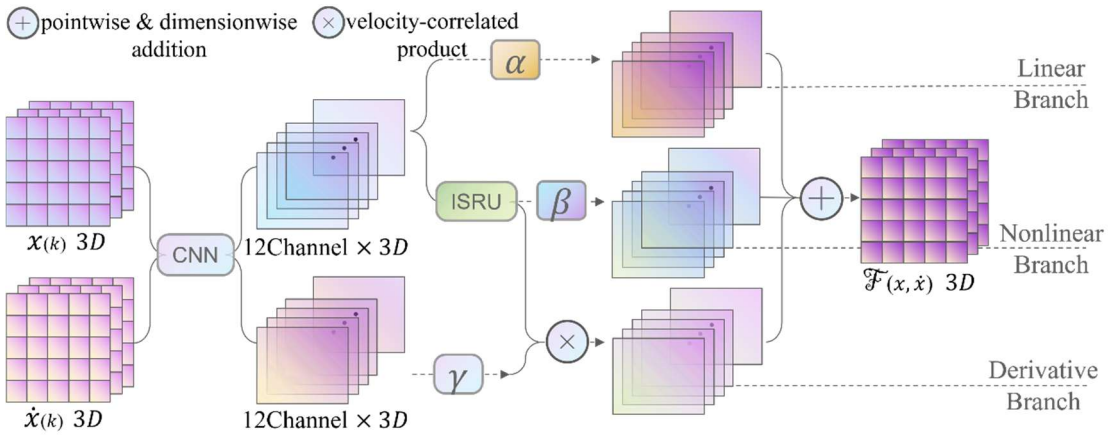
Code repository：https://github.com/Furkath/DRL_controlled_fluid-rigid_simulation
Research Report：
https://github.com/Furkath/DRL_controlled_fluid-rigid_simulation/blob/master/Document.pdf

6. Fabric Simulation by ML Model with Embedded Physics

My research topic is AI-empowered PDE system solving algorithms. My group has long been engaged in research on the application of machine learning in the simulation of thermal fluids and viscoelastic objects. Inspired by the work of applied mathematicians in Physics-informed-neural-networks, PDE-Nets, and Physics-recurrent-convolutional-neural-networks in recent years, all general PDE systems (including physics-based animation simulations) can adopt non-data-driven physical residuals as the optimization target for the learning process. Related work in the field of computer graphics can also be seen in TOG's Neural Cloth, etc. Nevertheless, typical physical problems have very obvious characteristics in spatial interaction and time progression, which can optimize network structure to avoid the need for general models to increase depth to fit data, thereby causing difficulties in training, convergence, and reasoning. Therefore, I try to use a network structure that includes CNN (analyzing spatial interaction) and embedded physical features (simulating the nonlinear relationship of various forces) to learn the stepping of spring-mass systems in explicit integration. The code is written in Pytorch and Taichi, and preliminary results have been demonstrated, with NN and PBS effects being similar. NN parameters usually have higher degrees of freedom than real physical parameters, but there is no overfitting in this Priori Test. This article was published in a CS conference, and further optimization of CNN's parameter quantity and structural features will be conducted later. Attempts will be made to add learning of cloth self-collision and implicit integration to this DL framework.

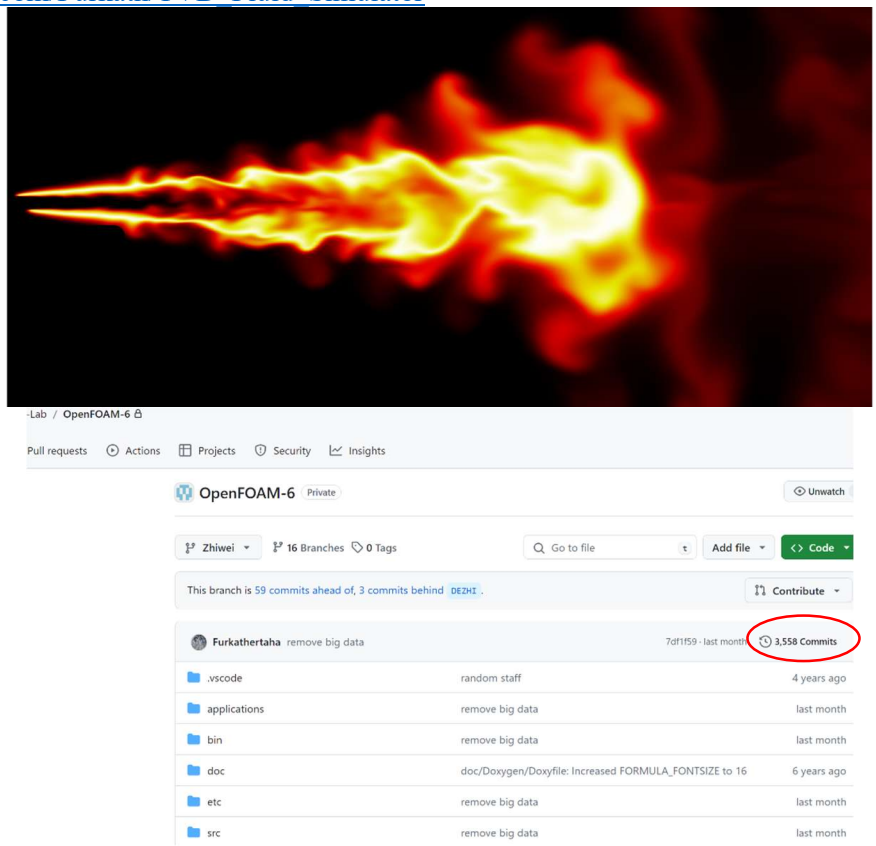
Code repository：https://github.com/Furkath/DL_Framework-for-PBS-Cloth-Simulation





7. Multi-scale Simulation

Choosing the simulation of flames as a representative of multi-scale PDE systems, typical gas-phase fluid simulations generally use mesh division combined with the finite volume method. Therefore, 3D CFD large eddy simulation can serve as a foundation for developing a series of acceleration algorithms that sacrifice the accuracy of small-scale features. These include topology, low-dimensional manifolds, mechanism simplification, Hash dynamic tables, digital twins, and AI data-driven methods. Numerous studies on Fourier Neural Operators (FNO) have also been conducted, with details available in the subsequent project team achievements. I am also responsible for the construction and maintenance of the project team's server, familiar with UNIX command shell principles, as well as system configuration files and various compilation tools. High-precision physical simulations can serve as a baseline for multiple tasks, and the TVD format can be referenced at:
https://github.com/Furkath/TVD_Fluid_Simulator



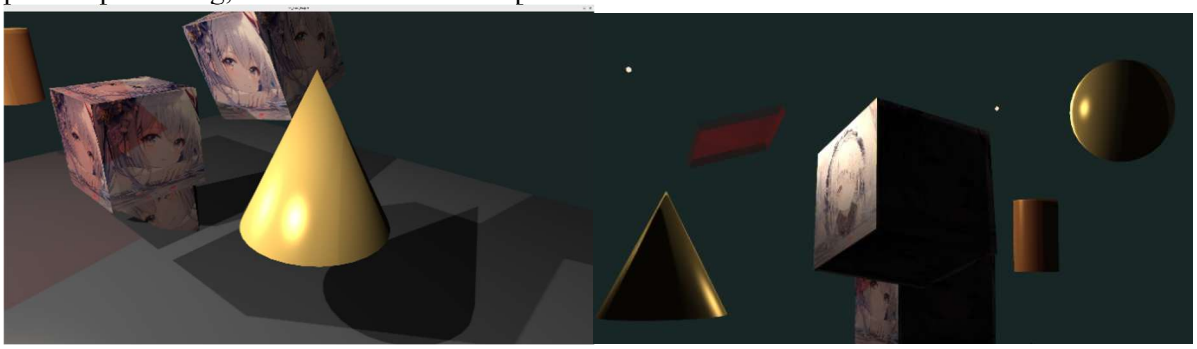
8. Self-docking of Unmanned Surface Vessels

The extension of the capstone design, involved RGBD camera sampling, 3D point cloud reconstruction, Minimum-Snap algorithm for shortest path planning, WIFI module ESP8266 point-to-point communication, L298N motor control, and Arduino microcontroller development. I was responsible for local wireless communication and microcontroller control development, assisting in the denoising algorithm for boat pose recognition, and building a Linux central C++ program based on timestamp stream processing for various tasks.

Code repository：https://github.com/Furkath/autoDock

9. OpenGL Illumination Model Rendering Practice

In this practice, I used the most basic C++ OpenGL API to complete the rendering of desired scenes. It read in Obj files and was coded with the BRDF lighting model in GLSL. I implemented dual-layer mapping, multiple light sources, Shadow Map shadows, translucency, supersampling, and texture optimization. Thoughts: The most basic OpenGL has typical C language characteristics, and the buffer process is quite cumbersome; GLSL ray tracing can try to pass the model as a texture to the GPU for parallel processing, which remains to be implemented.



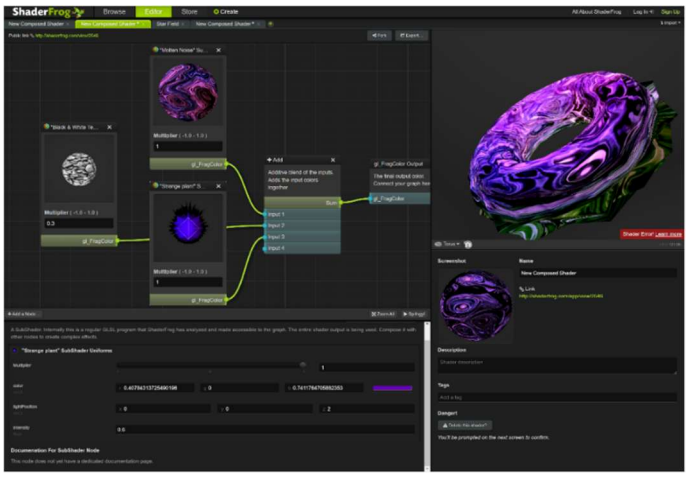
Code repository：https://github.com/Furkath/GL-Rendering

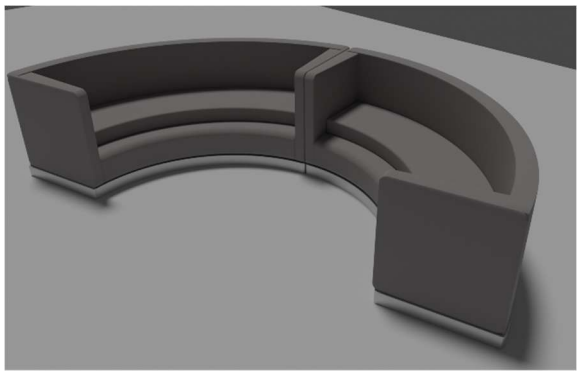
10. McGill University Entrepreneurship Program

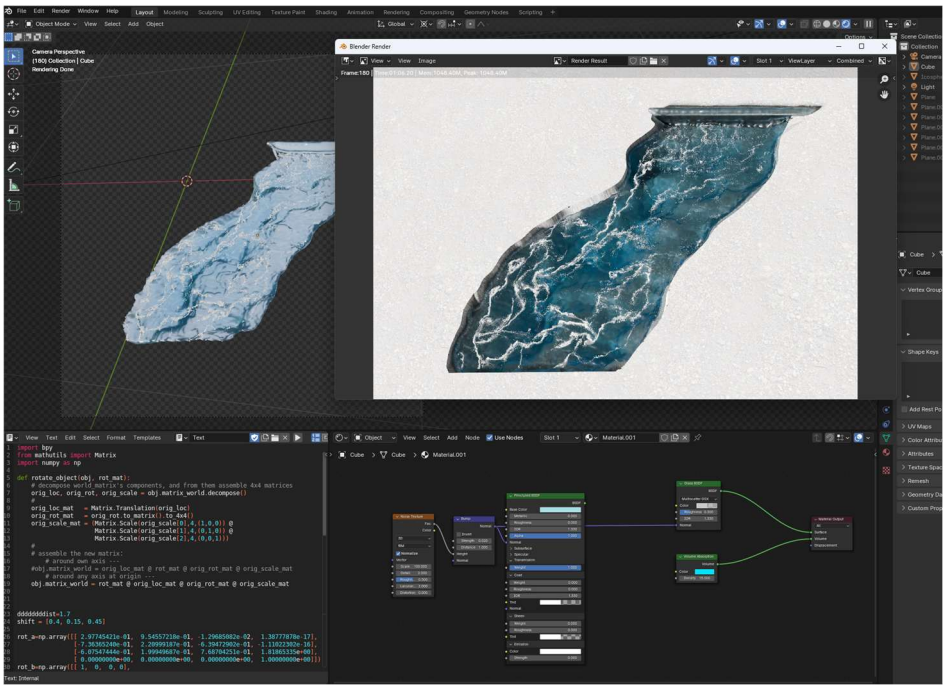
Participated in McGill's exchange program in business, conducted a simulated operation of a Canadian Chinese food delivery service in groups, as well as researched the business model of hockey sticks in the Canadian market in the last century.

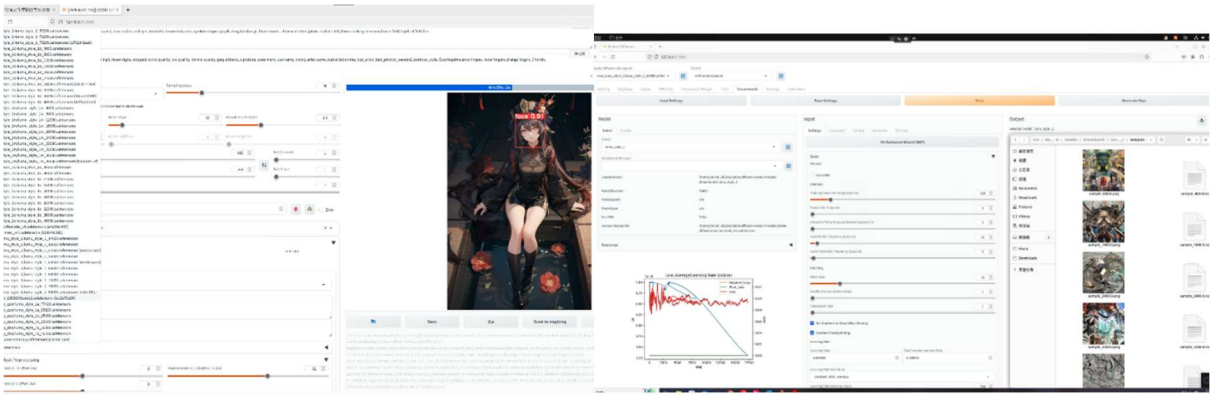
11. Personal Interests

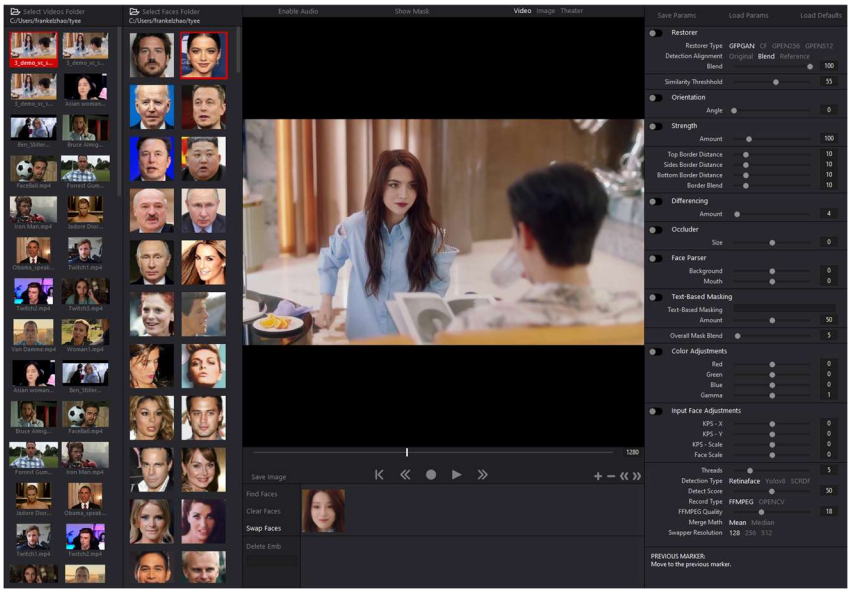
I have worked with some digital creation tools, including Blender, SD-WebUI, SD-ComfyUI, MMD, SolidWorks, Unity, UE, MD, ShaderFrog, RenderMan (Aqsis), etc. Among them, I have experience in reviewing the source code of Blender (to copy some mature open-source software graphics algorithms), using geometry nodes, and calling Python API scripts for procedural modeling. I have experience deploying SD-WebUI on ROCm's AMD cards, adding new types of samplers to the source code, and rolling back and resolving version conflicts to repeat the model training process. Particularly with ComfyUI, I have practiced the workflow of video face-swapping.

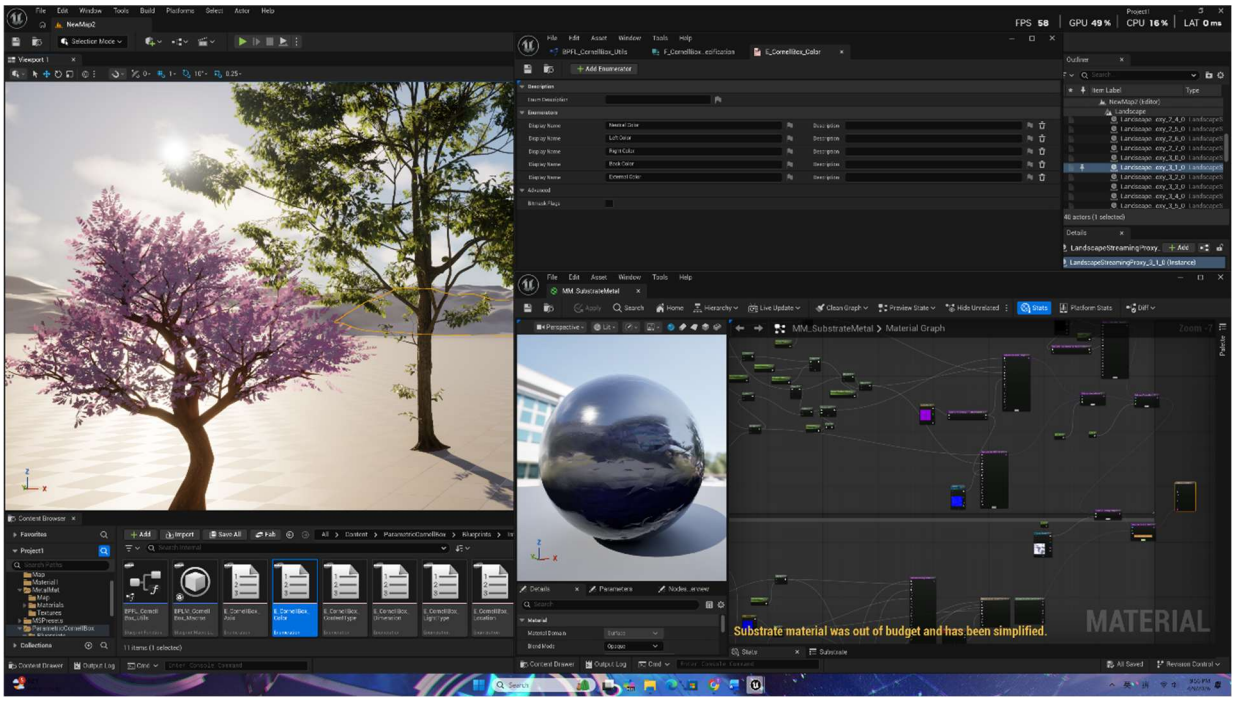





Blender Render
import bpy
import numpy as np
def rotate_object(obj, rot_mat):
shift = [0.4, 0.15, 0.45]




Select Videos Folder
Select Faces Folder
Enable Audio
Show Mask
Video
Image
Theater
Save Params
Load Params
Load Defaults
Restorer
Orientation
Strength
Differencing
Occluder
Face Parser
Text-Based Masking
Color Adjustments
Input Face Adjustments
Save Image
Find Faces
Clear Faces
Swap Faces
Delete Emb


FPS 58
GPU 49 %
CPU 16 %
LAT 0 ms
MM_SubstrateMetal > Material Graph
Substrate material was out of budget and has been simplified.
MATERIAL